

Report: Admin Account Takeover via JWT Algorithm Confusion (WEB API-Attack)

Course: GDT3CR – Ethical Hacking

Author: Samuel Melin

Date: 2026-03-11

1. Goal

This project demonstrates a Web **API attack** specifically targeting the authentication layer. By exploiting a flaw in the JWT validation logic of the REST API, we can bypass identity verification and perform unauthorized actions on administrative API endpoints.

2. Summary

In this project, we identify that the web application trusts the "none algorithm" in the JWT header. We use this to forge a token that identifies us as the administrator. With this forged token, we access the "Change Password" functionality to reset the administrator's password, effectively taking full control of the account.

3. Background

- **JWT Structure:** A token consisting of a Header, Payload, and Signature (Okta, 2026). The server uses the signature to verify that the payload hasn't been tampered with.
- **None-Algorithm:** A weakness where the server is tricked into not checking the signature because the header claims no signature is present ("alg: none") (De Ryck, 2021).
- **Privilege Escalation:** By manipulating the information in the token, an attacker can assume the identity of a higher-privileged user or anyone else that we have the account information from.

4. Prerequisites

- Install OWASP Juice Shop via Docker.
- Configure **Kali Linux** with **Burp Suite**.
- Familiarize yourself with the "Change Password" HTTP request structure in Juice Shop.
- JWT tool or using the website <http://jwt.io> (<http://jwt.io>).

5. Setup

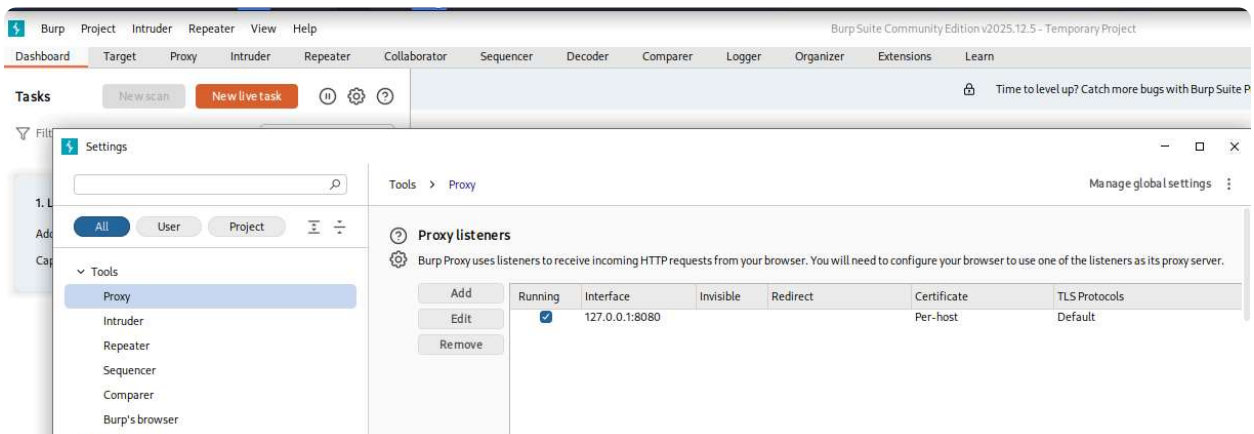
1. Launch the Juice Shop container and access it via your Kali browser:

```
(kali@kali)-[~]
└─$ sudo docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
bkimminich/juice-shop latest      ca116678ec45 3 days ago    490MB
hello-world         latest     1b44b5a3e06a 7 months ago  10.1KB

(kali@kali)-[~]
└─$ sudo docker run -d -p 4000:3000 --name juice-shop bkimminich/juice-shop
4369f42ec6ed947c15ebe623d9fa3b30130d25a9365a0d7e4e87c18a78aff64b

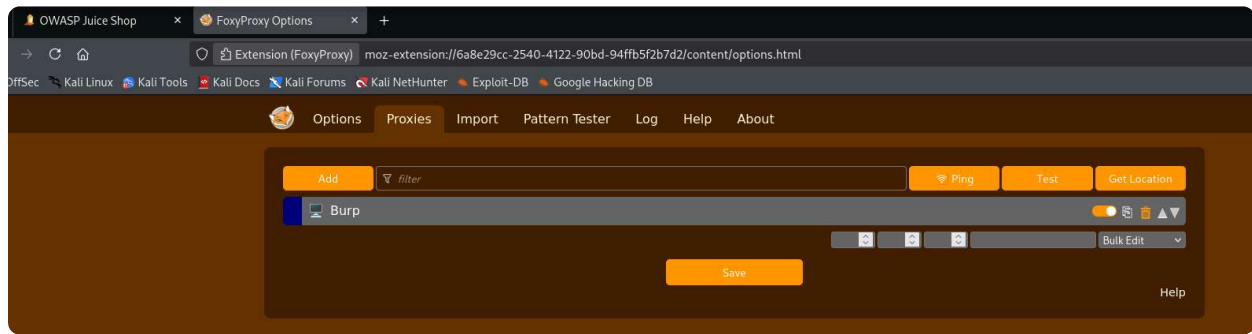
(kali@kali)-[~]
└─$ sudo docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                               NAMES
4369f42ec6ed   bkimminich/juice-shop "/nodejs/bin/node /j_..." 5 seconds ago  Up 4 seconds  0.0.0.0:4000→3000/tcp, [::]:4000→3000/tcp  juice-shop
```

2. Open Burp Suite and ensure the **Proxy** is listening on 127.0.0.1:8080 .

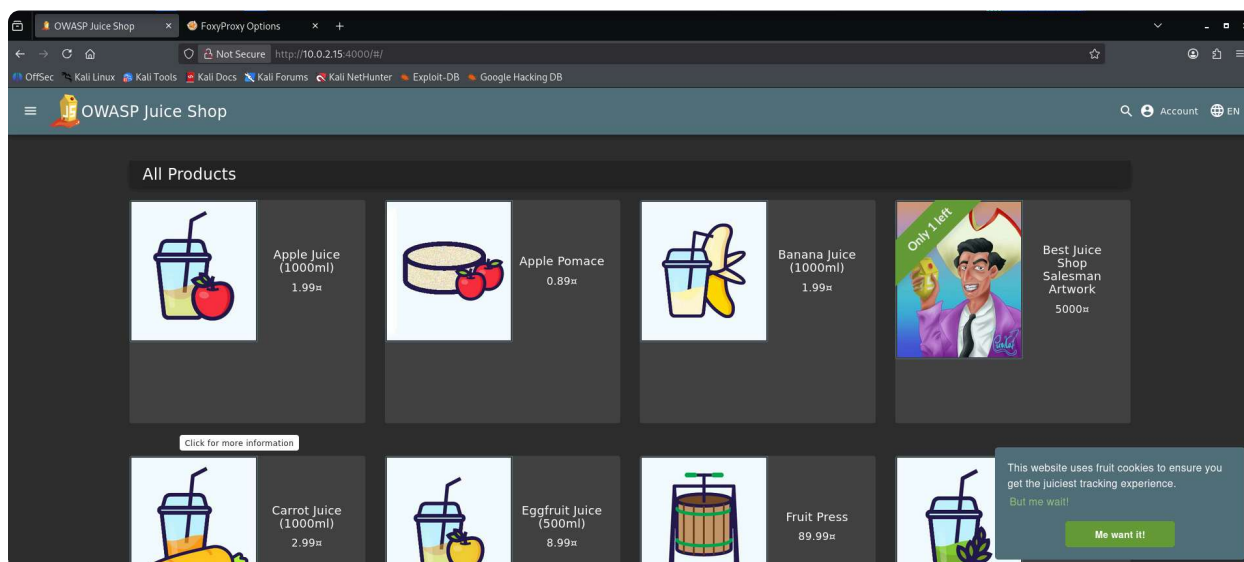


I used foxyproxy for firefox but you could use the in build browser with burp

aswell.



3. Register a standard user account and log in to generate a valid session JWT.



User Registration

Email*
bengt@bengt.com

Password*
●●●●●●
ⓘ Password must be 5-40 characters long. 6/20

Repeat Password*
●●●●●●
6/40

☒ Show password advice

Security Question*
Mother's maiden name? ▼
ⓘ This cannot be changed later!

Answer*
none

 Register

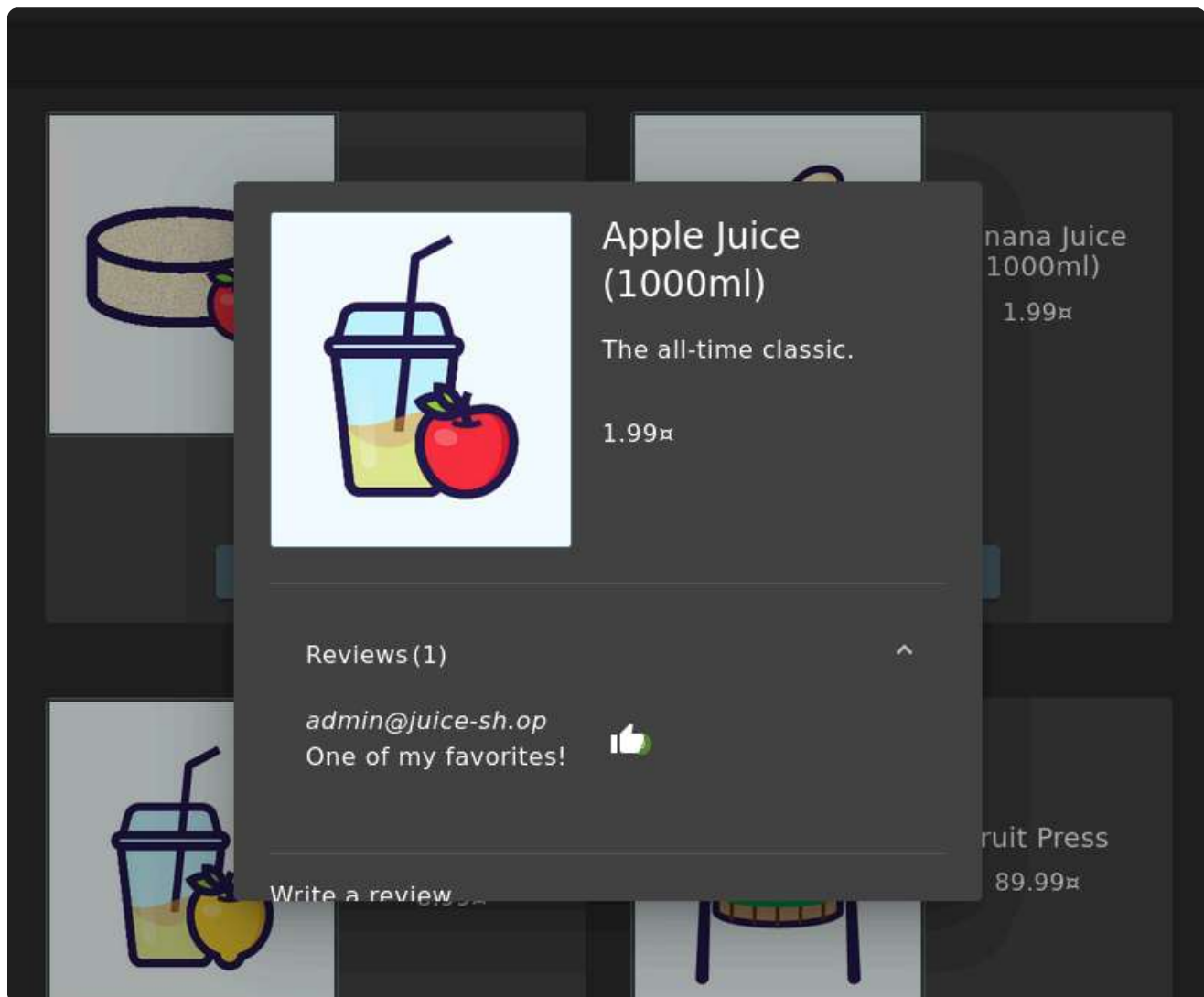
Already a customer?

I set the password on bengt@bengt.com (mailto:bengt@bengt.com) account to 123456.

5. Scanning.

1. Scan the website for emails address for the workers.

In this case we found a review from the admin@juice-sh.op which is a perfect target:



6. Exploitation (Account Takeover)

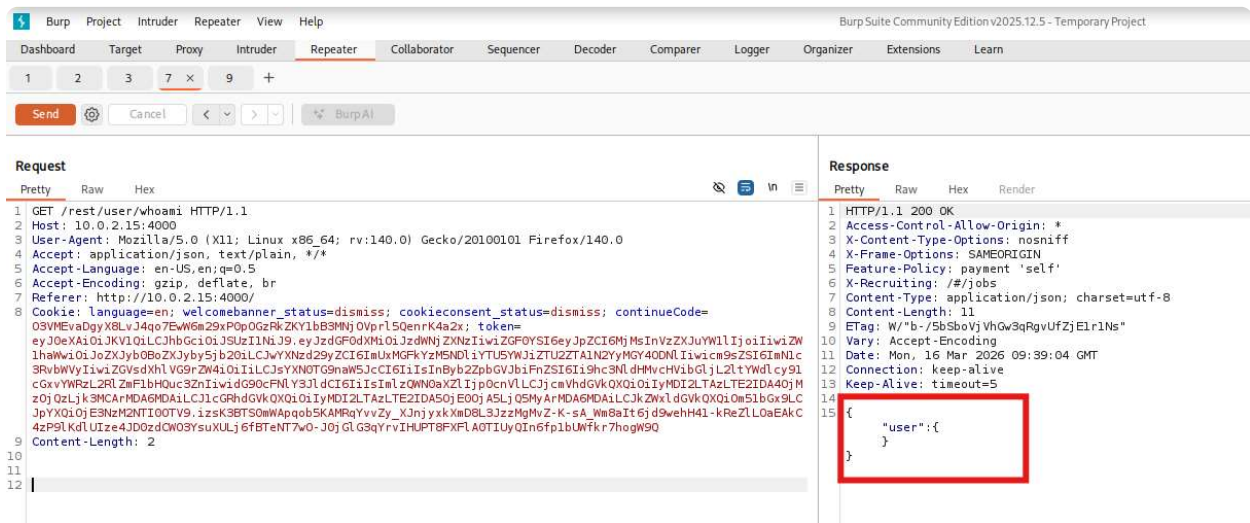
1. **Capture the JWT:** Login to your bengt@bengt.com (mailto:bengt@bengt.com) account. In (<http://account.in>) burp suite open the POST request `"/rest/user/login"`:

The screenshot shows the network tab of a web browser's developer tools. The 'Request' tab is active, displaying the raw HTTP request for a POST to /rest/user/login. The request includes headers like Host, User-Agent, Accept, and Content-Type, and a body with email and password. The 'Response' tab is also visible, showing a 200 OK status and a JSON body with an authentication token.

[illegible]

- This can be the most time consuming step where you need to try to find a solution where the "rest/user/whoami" response with a empty object.

- Send the "rest/user/whoami" request to repeater and change the token with a letter in payload for the bearer and the cookie.
- Try modifying the request until you get a response like this:

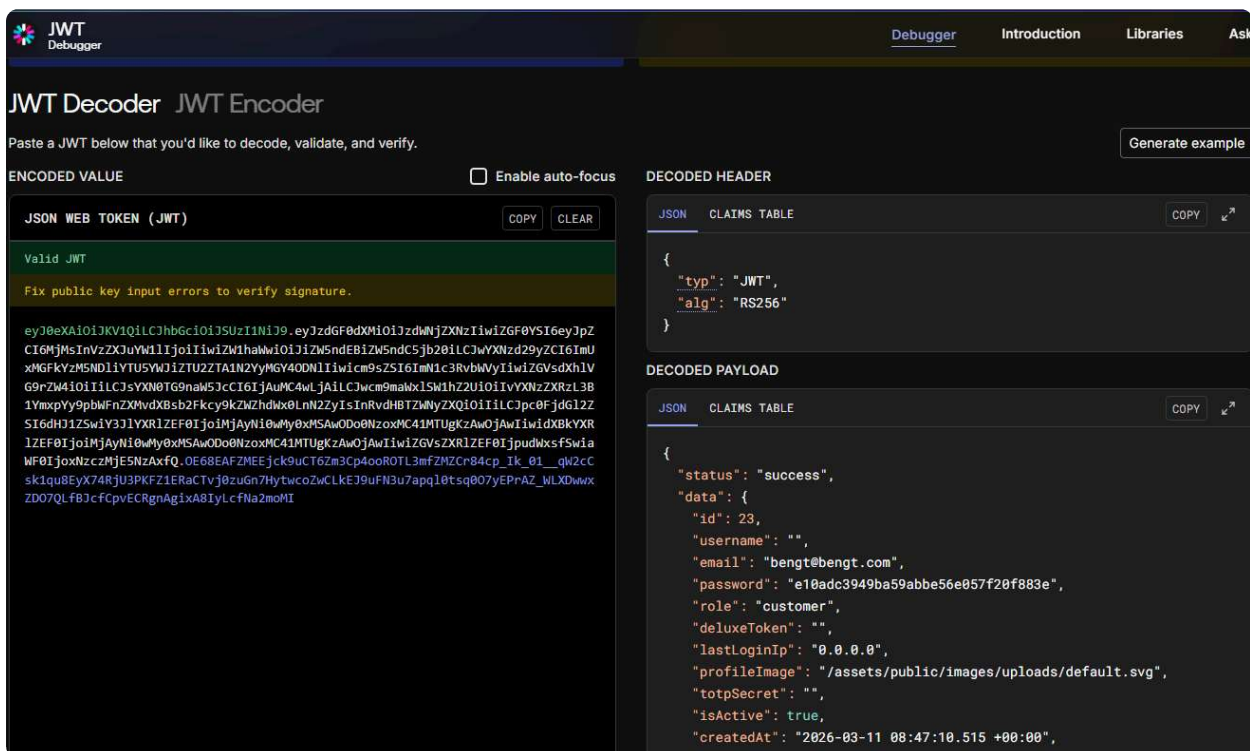


The screenshot shows the Burp Suite Repeater interface. On the left, the 'Request' tab is active, displaying a GET request to `/rest/user/whoami` with various headers and a cookie containing a token. On the right, the 'Response' tab is active, showing a 200 OK status with headers like `Access-Control-Allow-Origin` and a JSON body. A red box highlights the JSON body, which contains a `user` object.

This cookie will be used for later step.

3. Forge the Admin Token:

- Take your current JWT and decode it (use <http://jwt.io> (<http://jwt.io>)):



The screenshot shows the JWT Decoder tool interface. The 'ENCODED VALUE' field contains a JWT token. The 'DECODED HEADER' section shows the header structure with `"typ": "JWT"` and `"alg": "RS256"`. The 'DECODED PAYLOAD' section shows the payload structure, which includes fields like `"status": "success"`, `"data": { "id": 23, "username": "", "email": "bengt@bengt.com", "password": "e10adc3949ba59abbe56e057f20f883e", "role": "customer", "deluxeToken": "", "lastLoginIp": "0.0.0.0", "profileImage": "/assets/public/images/uploads/default.svg", "totpSecret": "", "isActive": true, "createdAt": "2026-03-11 08:47:10.515 +00:00", "updatedAt": "2026-03-11 08:47:10.515 +00:00" }`.

- **Modify Header:** Change `"alg": "HS256"` to `"alg": "none"` :

- **Modify Payload:** Change the email to admin@juice-sh.op (or the specific admin email) and set "role": "admin" and "id": "1" (we assume that admin has ID 1 here since ID points to the amount of users on this web-application):

The screenshot shows the JWT Debugger interface. The 'HEADER: ALGORITHM & TOKEN TYPE' section is valid and contains:


```
{
  "typ": "JWT",
  "alg": "none"
}
```

 The 'PAYLOAD: DATA' section is also valid and contains a JSON object with user details:


```
{
  "status": "success",
  "data": {
    "id": 1,
    "username": "",
    "email": "admin@juice-sh.op",
    "password": "e10adc3949ba59abbe56e057f20f883e",
    "role": "admin",
    "deluxeToken": "",
    "lastLoginIp": "0.0.0.0",
    "profileImage": "/assets/public/images/uploads/default.svg",
    "totpSecret": "",
    "isActive": true,
    "createdAt": "2026-03-11 08:47:10.515 +00:00",
    "updatedAt": "2026-03-11 08:47:10.515 +00:00",
    "deletedAt": null
  },
  "iat": 1773219701
}
```

 The 'JSON WEB TOKEN' section shows the resulting token as a long Base64 string, indicating it is an unsecured JWT.

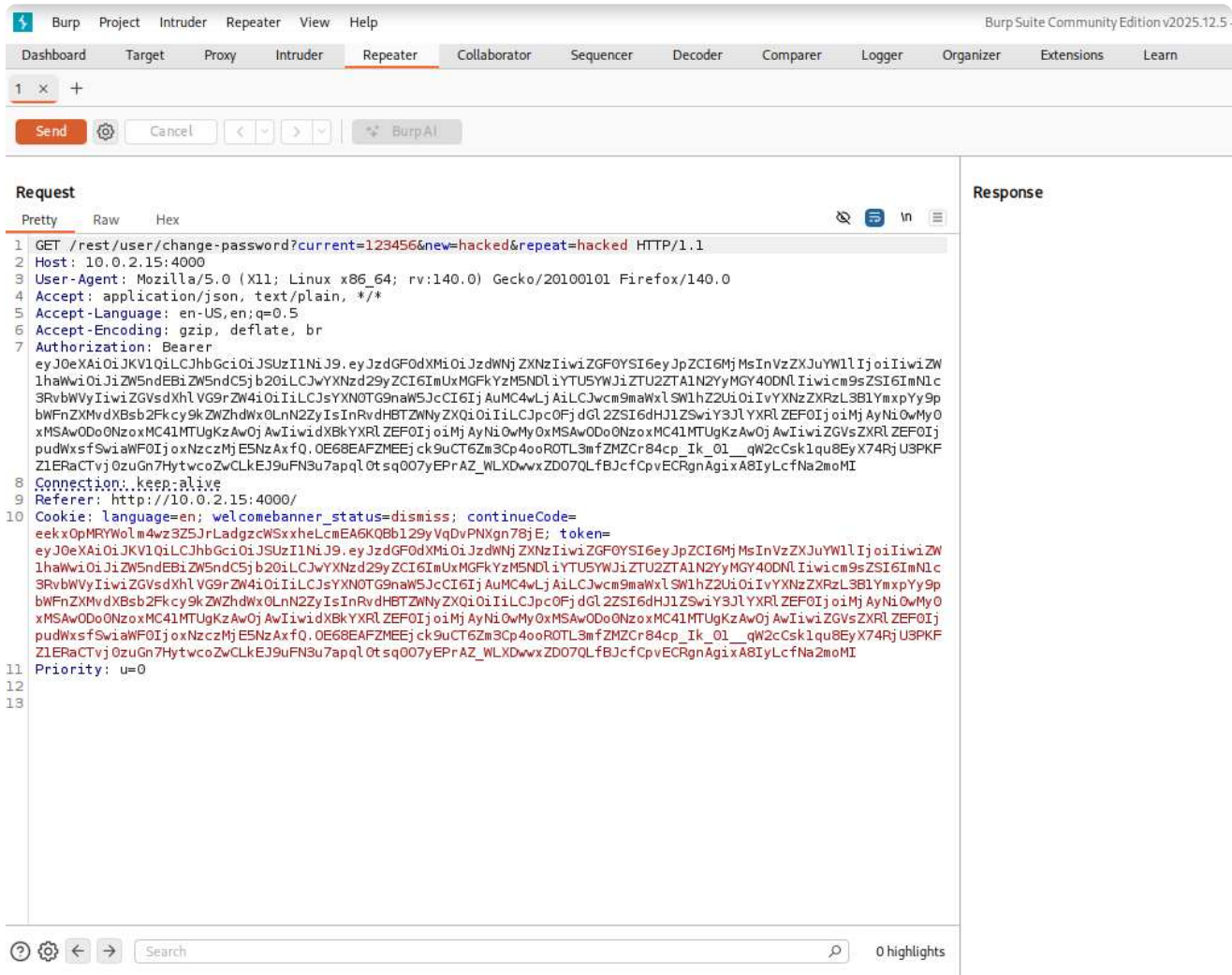
4. **The "None" Signature:** Re-encode the sections to Base64. Construct the token as header.payload. (ensuring the final dot is present but the signature part is empty).

5. Execute the Change:

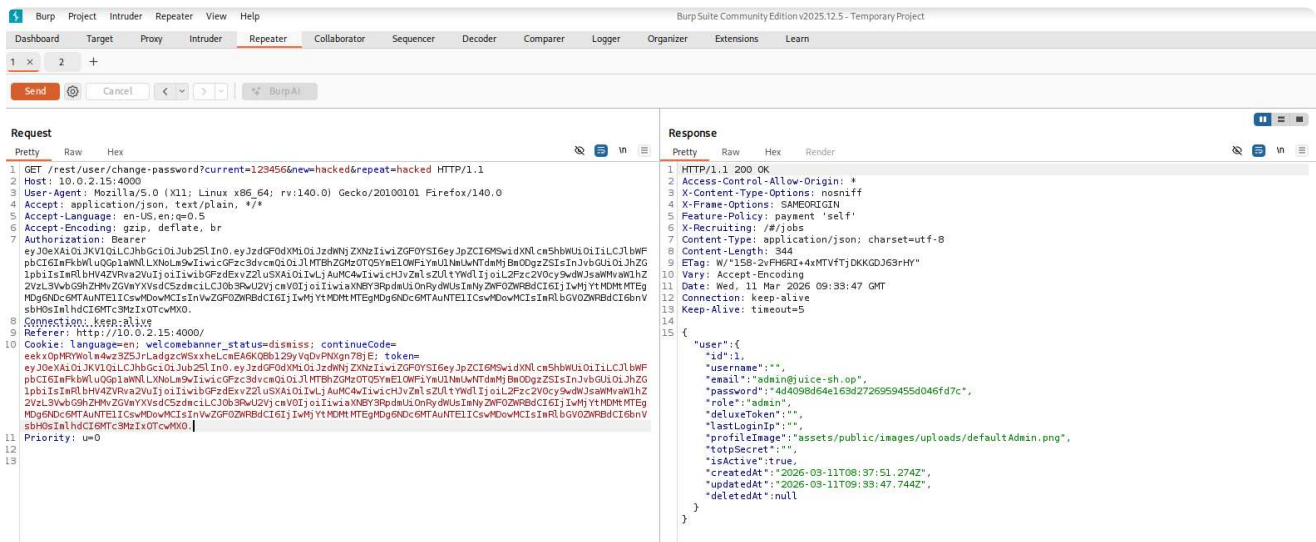
- Go to your juice shop account and change password.
I changed from 123456 to hacked:

- Go to burp and capture the changepassword request send it to a repeater:

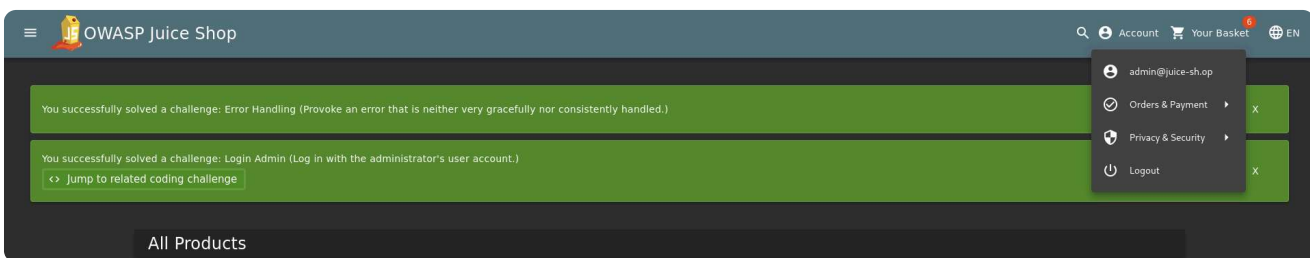
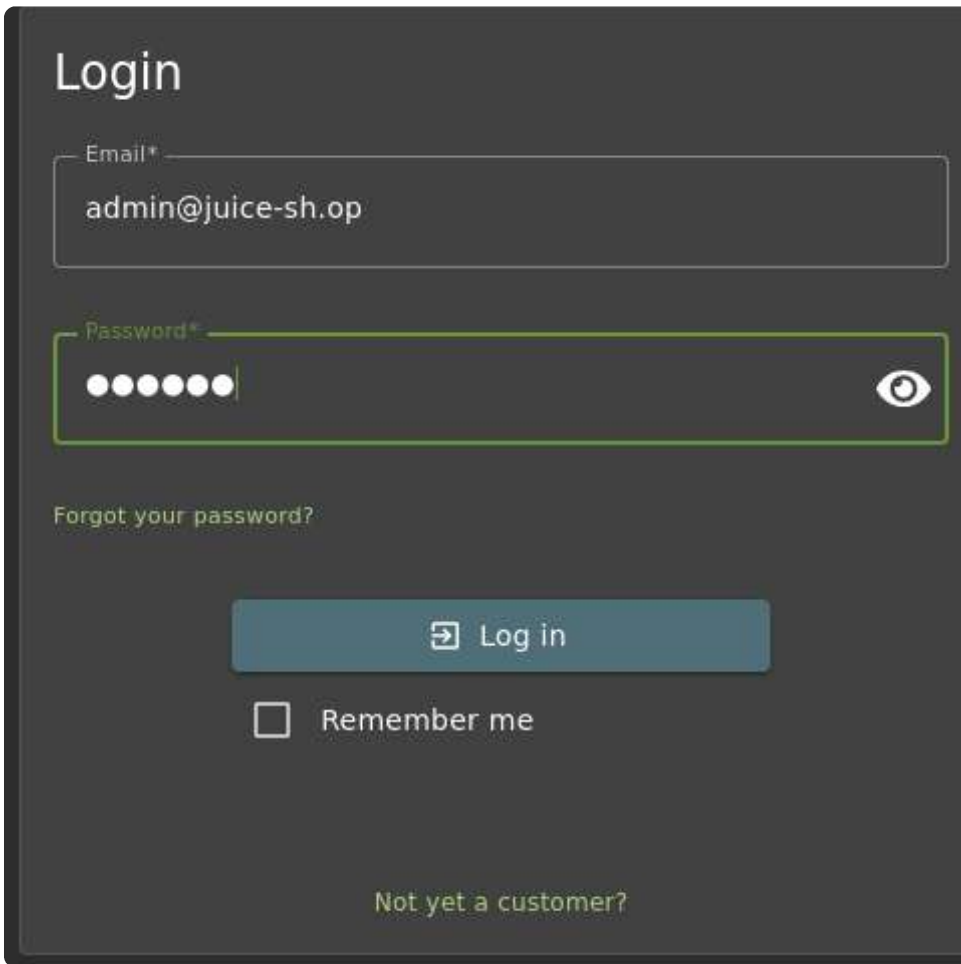




- In Burp **Repeater**, use the forged Admin JWT in the "Authorization: Bearer" and use the complete Cookie token from step 2 that was in the repeater:



5. **Verify Access:** Log out and log back in using the administrator's email and the new password you just set:



7.0 Prevention

How do you prevent this specific attack?

- By using a "blocklist" or "allowlist" for algorithms at the application level, ensuring that the `none` algorithm is strictly rejected during token validation* (OWASP Foundation, 2026).
- Implement short expiration times. Ensure that even if a token is stolen or manipulated, it will quickly become invalid, limiting the attacker's window of opportunity (De Ryck, 2021).

8.0 References

- De Ryck, P. (2021). *JWT Security. Pragmatic Web Security*.
https://owasp.org/www-chapter-vancouver/assets/presentations/2020-01_Attacking_and_Securing_JWT.pdf (https://owasp.org/www-chapter-vancouver/assets/presentations/2020-01_Attacking_and_Securing_JWT.pdf).
- Okta. (2026). *Introduction to JSON Web Tokens*. *JWT.io* (<http://JWT.io>).
<https://jwt.io/introduction> (<https://jwt.io/introduction>).
- OWASP Foundation. (2026). *Testing JSON Web Tokens (WSTG-SESS-10)*.
OWASP Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens
(https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens).